Day 13: Merkle Trees, Data Integrity & Anti-Entropy

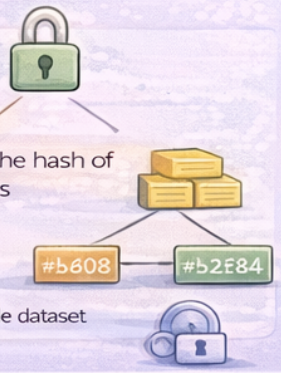
Comparing large datasets efficiently without transferring every record.

Why Merkle Trees Matter

- Large replicas are expensive to compare record by record.
- A Merkle tree summarizes a dataset into a small **root hash**
- If two replicas have the same **root**, their data matches for same ordering & hash rules.

Tree Structure

- Leaves store hashes of raw values.
- Internal nodes store the hash of their two child hashes combined together
- The topmost node is the "root hash", which represent the whole dataset



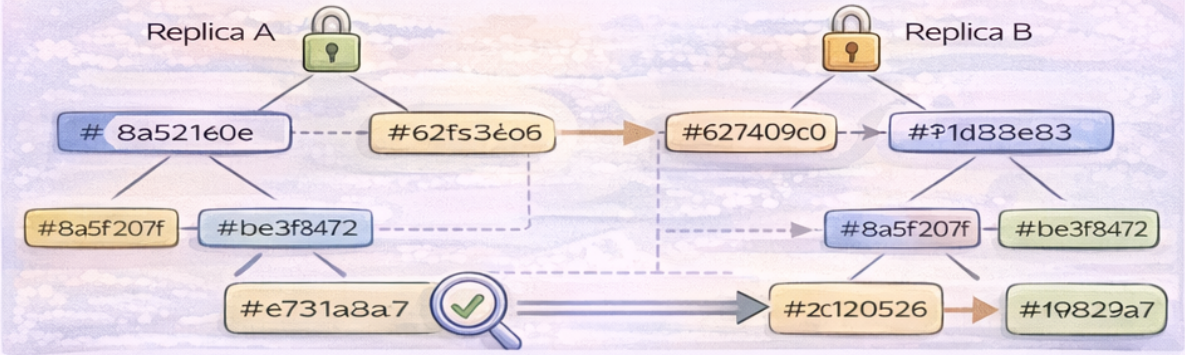
Membership Proofs

- A Merkle proof contains sibling hashes needed to recompute the root from one leaf
- This verifies one value belongs to a larger dataset without seeing every item
- Proofs are useful when trust, bandwidth, or storage is limited

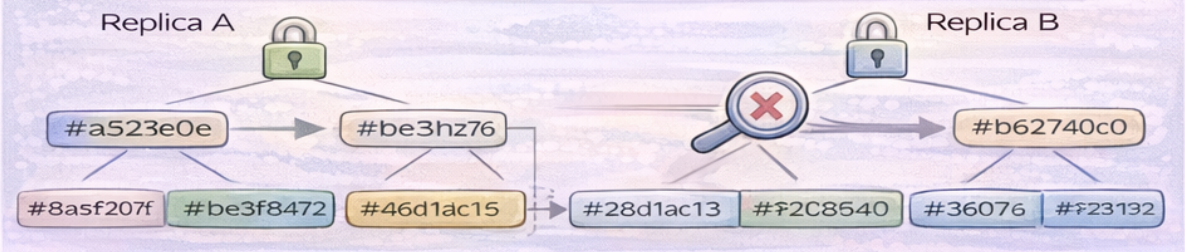
Anti-Entropy & Synchronization

- Replicas compare root hashes
- If roots differ, narrow mismatch to subtrees next.
- Compare sub-hashes until mismatches are isolated

Sample Merkle Tree with Replica Comparison



Sample Merkle Tree with Replica Comparison



Merkle trees provide compact proots & efficient sync for large, distributed datasets due to their hierarchical hashing design.

Overview from README.md

Day 13: Merkle Trees, Data Integrity, and Anti-Entropy

Day 13 focuses on how distributed systems compare large datasets efficiently without transferring every record.

It covers:
- Merkle trees and hierarchical hashing
- root hashes and integrity checks
- membership proofs
- replica comparison and anti-entropy
- synchronization in storage and blockchain-like systems
- hashing trade-offs and collision considerations

In short, Day 13 is about using tree-shaped hashes to detect divergence, verify inclusion, and synchronize distributed data efficiently.

![Day 13](./Day13.png)

[Day 13 PDF](./System_Design_Day_13.pdf)

Core ideas

Why Merkle trees matter
- Large replicas are expensive to compare record by record.
- A `Merkle tree` summarizes a dataset into a small root hash.
- If two replicas have the same root, their data matches for the same ordering and hash rules.

Tree structure
- Leaves store hashes of raw values.
- Internal nodes store the hash of their two child hashes combined together.
- The topmost node is the `root hash`, which represents the whole dataset.

Membership proofs
- A `Merkle proof` contains the sibling hashes needed to recompute the root from one leaf.
- This lets a client verify that one value belongs to a larger dataset without seeing every item.
- Membership proofs are useful when trust, bandwidth, or storage is limited.

Anti-entropy and synchronization
- Replicas can compare root hashes first.
- If roots differ, they can compare subtree hashes to narrow the mismatch quickly.
- This avoids moving the full dataset when only a small subset changed.

Trade-offs
- Tree results depend on deterministic ordering of input values.
- Updates can require recomputing hashes along the changed path.
- Hash choice matters for speed, security, and interoperability.

Day-13 sample: Merkle Tree

This repository includes a small Merkle tree sample in both Python and Java.

File: demo.py

```python
from merkle_tree import MerkleTree


def print_proof(label: str, proof: list) -> None:
    print(label)
    for step in proof:
        side = "left" if step.sibling_on_left else "right"
        print(f"  sibling on {side}: {step.sibling_hash[:16]}...")


def main() -> None:
    replica_a = ["order-1001", "order-1002", "order-1003", "order-1004"]
    replica_b = ["order-1001", "order-1002", "order-9999", "order-1004"]

    tree_a = MerkleTree(replica_a)
    tree_b = MerkleTree(replica_b)

    print("replica A root:", tree_a.root_hash)
    print("replica B root:", tree_b.root_hash)
    print("roots match:", tree_a.root_hash == tree_b.root_hash)

    proof = tree_a.proof_for(2)
    print_proof("\nproof for replica A value order-1003", proof)

    verified = MerkleTree.verify_proof("order-1003", proof, tree_a.root_hash)
    print("\nproof verifies against replica A root:", verified)

    mismatch = MerkleTree.verify_proof("order-1003", proof, tree_b.root_hash)
    print("same proof verifies against replica B root:", mismatch)


if __name__ == "__main__":
    main()
```

File: merkle_tree.py

```python
from __future__ import annotations

import hashlib
from dataclasses import dataclass


@dataclass(frozen=True)
class ProofStep:
    sibling_hash: str
    sibling_on_left: bool


class MerkleTree:
    def __init__(self, values: list[str]) -> None:
        if not values:
            raise ValueError("values must be non-empty")
        if any(not value for value in values):
            raise ValueError("values must not contain empty items")

        self._values = values[:]
        self._levels = self._build_levels(values)

    @property
    def root_hash(self) -> str:
        return self._levels[-1][0]

    def proof_for(self, index: int) -> list[ProofStep]:
        if index < 0 or index >= len(self._values):
            raise IndexError("index out of range")

        proof: list[ProofStep] = []
        current_index = index

        for level in self._levels[:-1]:
            sibling_index = current_index - 1 if current_index % 2 else current_index + 1
            if sibling_index >= len(level):
                sibling_index = current_index

            proof.append(
                ProofStep(
                    sibling_hash=level[sibling_index],
                    sibling_on_left=sibling_index < current_index,
                )
            )
            current_index //= 2

        return proof

    @staticmethod
    def verify_proof(value: str, proof: list[ProofStep], expected_root: str) -> bool:
```

```
        if not value:
            raise ValueError("value must be non-empty")

        current_hash = MerkleTree._leaf_hash(value)
        for step in proof:
            if step.sibling_on_left:
                current_hash = MerkleTree._parent_hash(step.sibling_hash, current_hash)
            else:
                current_hash = MerkleTree._parent_hash(current_hash, step.sibling_hash)
        return current_hash == expected_root

    @classmethod
    def _build_levels(cls, values: list[str]) -> list[list[str]]:
        levels: list[list[str]] = [[cls._leaf_hash(value) for value in values]]

        while len(levels[-1]) > 1:
            current = levels[-1]
            next_level: list[str] = []
            for index in range(0, len(current), 2):
                left = current[index]
                right = current[index + 1] if index + 1 < len(current) else current[index]
                next_level.append(cls._parent_hash(left, right))
            levels.append(next_level)

        return levels

    @staticmethod
    def _leaf_hash(value: str) -> str:
        return hashlib.sha256(f"leaf:{value}".encode("utf-8")).hexdigest()

    @staticmethod
    def _parent_hash(left: str, right: str) -> str:
        return hashlib.sha256(f"node:{left}:{right}".encode("utf-8")).hexdigest()
```

File: Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> replicaA = List.of("order-1001", "order-1002", "order-1003", "order-1004");
        List<String> replicaB = List.of("order-1001", "order-1002", "order-9999", "order-1004");

        MerkleTree treeA = new MerkleTree(replicaA);
        MerkleTree treeB = new MerkleTree(replicaB);

        System.out.println("replica A root: " + treeA.getRootHash());
        System.out.println("replica B root: " + treeB.getRootHash());
        System.out.println("roots match: " + treeA.getRootHash().equals(treeB.getRootHash()));

        List<MerkleTree.ProofStep> proof = treeA.proofFor(2);
        printProof("\nproof for replica A value order-1003", proof);

        boolean verified = MerkleTree.verifyProof("order-1003", proof, treeA.getRootHash());
        System.out.println("\nproof verifies against replica A root: " + verified);

        boolean mismatch = MerkleTree.verifyProof("order-1003", proof, treeB.getRootHash());
        System.out.println("same proof verifies against replica B root: " + mismatch);
    }

    private static void printProof(String label, List<MerkleTree.ProofStep> proof) {
        System.out.println(label);
        for (MerkleTree.ProofStep step : proof) {
            String side = step.isSiblingOnLeft() ? "left" : "right";
            System.out.println("  sibling on " + side + ": " + step.getSiblingHash().substring(0, 16
) + "...");
        }
    }
}
```

File: MerkleTree.java

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.ArrayList;
import java.util.List;

public class MerkleTree {
    public static final class ProofStep {
        private final String siblingHash;
        private final boolean siblingOnLeft;

        private ProofStep(String siblingHash, boolean siblingOnLeft) {
            this.siblingHash = siblingHash;
            this.siblingOnLeft = siblingOnLeft;
        }

        public String getSiblingHash() {
            return siblingHash;
        }

        public boolean isSiblingOnLeft() {
            return siblingOnLeft;
        }
    }

    private final List<String> values;
    private final List<List<String>> levels;

    public MerkleTree(List<String> values) {
        if (values == null || values.isEmpty()) {
            throw new IllegalArgumentException("values must be non-empty");
        }
        for (String value : values) {
            if (value == null || value.isBlank()) {
                throw new IllegalArgumentException("values must not contain empty items");
            }
        }

        this.values = new ArrayList<>(values);
        this.levels = buildLevels(values);
    }

    public String getRootHash() {
        return levels.get(levels.size() - 1).get(0);
    }

    public List<ProofStep> proofFor(int index) {
        if (index < 0 || index >= values.size()) {
            throw new IndexOutOfBoundsException("index out of range");
        }
```

```
        List<ProofStep> proof = new ArrayList<>();
        int currentIndex = index;

        for (int levelIndex = 0; levelIndex < levels.size() - 1; levelIndex += 1) {
            List<String> level = levels.get(levelIndex);
            int siblingIndex = currentIndex % 2 == 0 ? currentIndex + 1 : currentIndex - 1;
            if (siblingIndex >= level.size()) {
                siblingIndex = currentIndex;
            }

            proof.add(new ProofStep(level.get(siblingIndex), siblingIndex < currentIndex));
            currentIndex /= 2;
        }

        return proof;
    }

    public static boolean verifyProof(String value, List<ProofStep> proof, String expectedRoot) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException("value must be non-empty");
        }

        String currentHash = leafHash(value);
        for (ProofStep step : proof) {
            if (step.isSiblingOnLeft()) {
                currentHash = parentHash(step.getSiblingHash(), currentHash);
            } else {
                currentHash = parentHash(currentHash, step.getSiblingHash());
            }
        }
        return currentHash.equals(expectedRoot);
    }

    private static List<List<String>> buildLevels(List<String> values) {
        List<List<String>> levels = new ArrayList<>();
        List<String> leaves = new ArrayList<>();
        for (String value : values) {
            leaves.add(leafHash(value));
        }
        levels.add(leaves);

        while (levels.get(levels.size() - 1).size() > 1) {
            List<String> current = levels.get(levels.size() - 1);
            List<String> nextLevel = new ArrayList<>();

            for (int index = 0; index < current.size(); index += 2) {
                String left = current.get(index);
                String right = index + 1 < current.size() ? current.get(index + 1) : current.get(ind
ex);
                nextLevel.add(parentHash(left, right));
            }
```

```
            levels.add(nextLevel);
        }

        return levels;
    }

    private static String leafHash(String value) {
        return sha256Hex("leaf:" + value);
    }

    private static String parentHash(String left, String right) {
        return sha256Hex("node:" + left + ":" + right);
    }

    private static String sha256Hex(String value) {
        try {
            byte[] digest = MessageDigest.getInstance("SHA-256").digest(value.getBytes(StandardChars
ets.UTF_8));
            StringBuilder builder = new StringBuilder(digest.length * 2);
            for (byte current : digest) {
                builder.append(String.format("%02x", current));
            }
            return builder.toString();
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 algorithm unavailable", e);
        }
    }
}
```